OPERATING SYSTEM
ORANGE 2 – PES-VCS LAB REPORT

Name: Supreeth Sharma Y V
SRN: PES1UG24CS635
Class: 4th Semester ‘K’ Section
Repository: https://github.com/supreeth-sharma/PES1UG24CS635-pes-vcs

Phase 1:
Screenshot 1A

```
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$
```

Screenshot 1B

```
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ git commit -am "Phase1: pass test_objects and finalize object store"
```

Phase 2:
Screenshot 2A

```
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$
```

Screenshot 2B

```
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a       blob 6.hello.
```

Phase 3:
Screenshot 3A

```
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ echo "hello" > file1.txt
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ echo "world" > file2.txt
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes add file1.txt file2.txt
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  tree.h
  untracked:  tree.c
  untracked:  index.c
  untracked:  .gitignore
  untracked:  hello.txt
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  commit.c
  untracked:  object.c
  untracked:  commit.h
  untracked:  bye.txt
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  Makefile

vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$
```

Screenshot 3B

```
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776323491 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776323491 6 file2.txt
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$
```

Phase 4:
Screenshot 4A

```
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ echo "Hello" > hello.txt
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes add hello.txt
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes commit -m "Initial commit"
Committed: ff6c1c352877... Initial commit
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ echo "World" >> hello.txt
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes add hello.txt
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes commit -m "Add world"
Committed: fb4b40a900b2... Add world
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ echo "Bye" > bye.txt
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes add bye.txt
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes commit -m "Add bye file"
Committed: f18c60e86fe8... Add bye file
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ ./pes log
commit f18c60e86fe8a523915a752edc2b2e1210beadf47c2a6dbee9a1a267c30fad09
Author: Supreeth Sharma Y V <PES1UG24CS635>
Date:   1776324214

    Add bye file

commit fb4b40a900b29ace9b2f9ccd38b9e54b04fc2a6453f6b09683c383b741070188
Author: Supreeth Sharma Y V <PES1UG24CS635>
Date:   1776324208

    Add world

commit ff6c1c35287745b57e1c02574fda05157786cc926dfc28749f8678bc2db0e43c
Author: Supreeth Sharma Y V <PES1UG24CS635>
Date:   1776324197

    Initial commit bye file
```

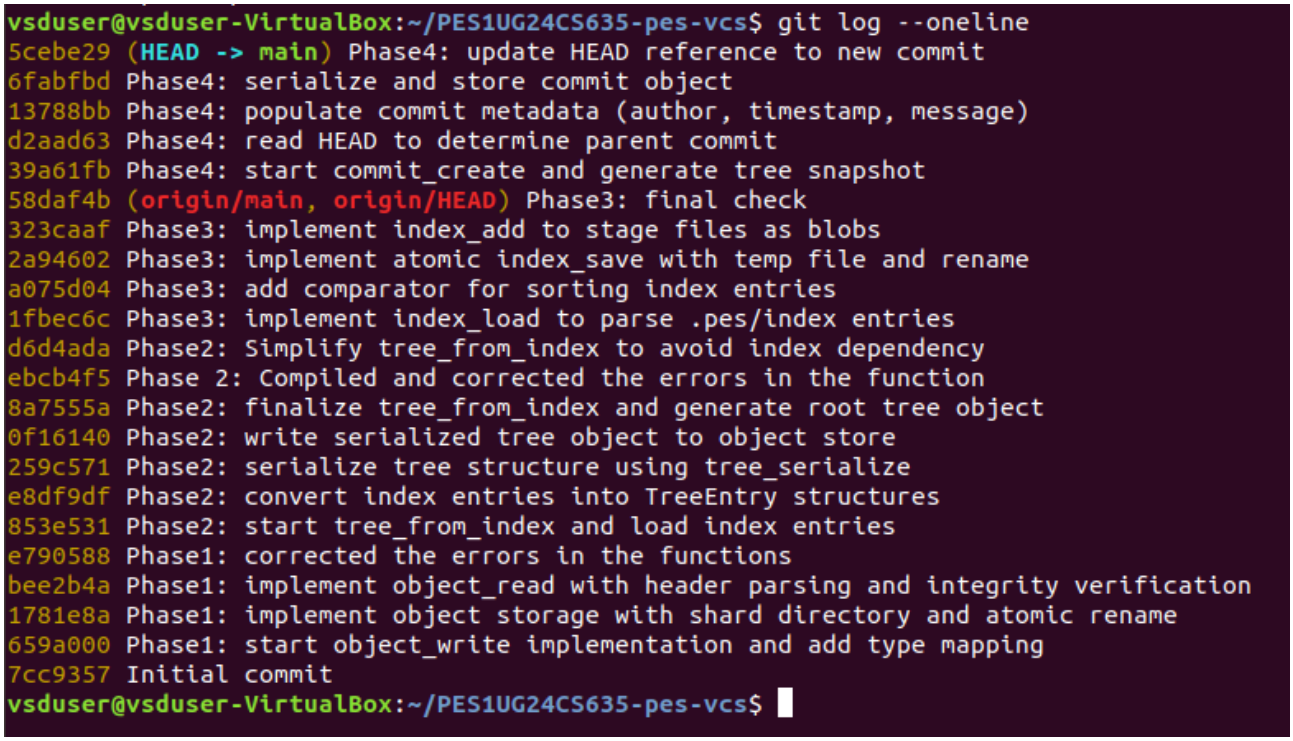
Screenshot 4B

```
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/01/63b63c78a66ff3aa41d80f22cd722dea7d08277186cc0a7cd0886eaecc6d24
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/3d/ef364f79a7f70b412a038acd2f11936cb6a8dc62434d1fbbfb853b4a9e6f8b
.pes/objects/48/1407f7237f7bd749dca31e2d8422bafe211741d5556f007e0faf821ffc7179
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/d7/8dc05696a742edcb4a66b9b82949080b40a3b63c480424d9ce846b40b1c2c5
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/f1/8c60e86fe8a523915a752edc2b2e1210beadf47c2a6dbee9a1a267c30fad09
.pes/objects/fb/4b40a900b29ace9b2f9ccd38b9e54b04fc2a6453f6b09683c383b741070188
.pes/objects/ff/6c1c35287745b57e1c02574fda05157786cc926dfc28749f8678bc2db0e43c
.pes/refs/heads/main
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$
```

Screenshot 4C

```
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ cat .pes/refs/heads/main
f18c60e86fe8a523915a752edc2b2e1210beadf47c2a6dbee9a1a267c30fad09
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
vsduser@vsduser-VirtualBox:~/PES1UG24CS635-pes-vcs$
```

Screenshot of all the commits made:



Phase 5:

Question 5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

In this system, a branch is simply a file inside `.pes/refs/heads/` that contains the hash of the latest commit on that branch. The `HEAD` file stores which branch is currently active. To implement the checkout command, the program would first locate the branch file corresponding to the branch name given by the user. The commit hash stored in that file represents the latest snapshot of that branch.

After reading this commit hash, the system would update the `HEAD` file so that it points to the selected branch. The program would then load the commit object associated with that hash and read the tree object referenced by the commit. The tree describes the structure of the project at that point in history.

The working directory must then be updated to match this snapshot. This means creating directories, restoring file contents from the object store, and removing files that do not belong to the target snapshot. Finally, the index should be updated so that it matches the checked-out state.

This operation is complex because the system must safely update many files while ensuring that existing work is not accidentally overwritten.

Question 5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

A dirty working directory means that some tracked files have been modified but those changes have not been committed yet. If the user attempts to switch branches while such modifications exist, those changes could be lost.

To detect this situation, the system can compare the working directory against the index. The index contains metadata about the last staged version of each tracked file, including its size and modification time. If a file in the working directory has a different size or timestamp than the entry stored in the index, it indicates that the file has been modified since it was last staged.

Next, the system must also compare the file against the version stored in the target branch’s commit tree. If the file differs between branches and the working copy is also modified, switching branches would overwrite the user’s work. In this situation the checkout operation should stop and warn the user that there are uncommitted changes that must be committed or discarded before switching branches.

Question 5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

A detached HEAD occurs when the HEAD file contains a commit hash directly instead of referencing a branch. Normally HEAD points to a branch name such as `refs/heads/main`, and that branch file stores the latest commit hash. In a detached state, HEAD refers directly to a specific commit instead of a branch.

If a user creates new commits while in this state, the commits will still be created normally and stored in the object database. However, no branch reference will point to them. Because there is no reference chain connecting those commits to the rest of the repository history, they may eventually become unreachable.

To recover such commits, the user can create a new branch that points to the detached commit. This simply involves creating a file in `.pes/refs/heads/` containing the commit hash and then

updating HEAD to reference that branch. Once the branch reference exists, the commits are safely connected to the repository history again.

Phase 6:

Question 6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Over time, a repository may contain objects that are no longer referenced by any commit or branch. These objects are considered unreachable and can safely be removed to save disk space.

To find such objects, the system would start by identifying all reachable commits. This can be done by reading every branch reference stored in `.pes/refs/heads/`. Each branch points to a commit hash. From each commit, the system would follow the parent links recursively until it reaches the earliest commit in that chain.

While traversing commits, the program would also record the tree object referenced by each commit. Every tree object contains references to blobs and possibly other trees. These references must also be followed recursively.

A practical way to track reachable objects is by using a hash set. Each object that is discovered during traversal is added to the set. Once traversal is complete, the system scans the entire `.pes/objects` directory. Any object whose hash is not present in the reachable set can be considered unused and may be deleted.

For a repository containing around one hundred thousand commits and several dozen branches, the traversal would still typically visit around the same number of commits because branches often share history. In addition to commits, the algorithm would also visit all tree and blob objects referenced by those commits.

Question 6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Running garbage collection at the same time as a commit operation can lead to serious race conditions. During a commit, new objects such as blobs, trees, and the commit object itself are written to the object store. However, these objects are not immediately referenced by any branch until the commit operation finishes and the branch reference is updated.

If garbage collection runs during this small window of time, it may scan the repository and conclude that these newly created objects are unreachable. Since they are not yet referenced by a commit chain or branch pointer, the garbage collector might delete them.

If that happens, the commit process will later attempt to reference objects that no longer exist in the object store, which would corrupt the repository.

Real version control systems avoid this situation by using careful synchronization techniques. They often prevent garbage collection from running while a commit is in progress, use temporary files followed by atomic renames, and delay deletion of newly created objects until they are safely referenced. These strategies ensure that objects are never removed while they are still part of an ongoing operation.